MuniPricer: Mid-Quarter Project Update

Anubhav Awasthi, Ibukun Agbaje and Eno Okon · BUS 43100 AI Essentials · November 4, 2026

Live app: munipricer.streamlit.app Code: github.com/anub-awasthi/munipricer Evidence packet attached

1. Problem definition

Nearly one million municipal bonds are outstanding, and on an average day in 2023 only about 2% of them traded (MSRB, 2024). If you hold one of the other 98%, nobody can tell you what it is worth today. Institutions get around this with evaluated pricing feeds from vendors like SOLVE and Tradeweb. A retired investor with a few muni bonds, or the treasurer of a small town or school district, does not have those feeds. Their only free reference is the last trade on EMMA, which can be months old and may include a dealer markup. So when they sell, they take the dealer's bid with no way to check it, and small trades are where markups are largest.

Our problem got narrower twice. In week 1 we wanted both an issuance assistant for municipalities and a secondary-market pricing tool. Working through the Lab 3 scaffold, the issuance half kept failing two questions: what decision changes, and what data do we have rights to. We dropped it in week 3. That same week, a competitor scan showed that SOLVE Px, Tradeweb AI-Price and Spline Data already price munis with machine learning at scale. We stopped thinking about beating them on accuracy and focused on the holder they do not serve: someone with one CUSIP who needs a number and a reason they can understand.

For a bond that has not traded recently, we define success three ways. The price should beat the last-trade lookup a holder uses today. The 80% range should contain the real price about 80% of the time. The explanation should only state what the model actually used. Our weak point here is that our grounding is secondhand. We have not yet talked to a treasurer, an individual muni investor or a dealer, and that is in our week 8 plan.

2. Approach and rationale

Pricing a bond that has not traded means inferring its value from thousands of trades in similar bonds. That is supervised learning on tabular data, which is where gradient-boosted trees are strongest. Our model (LightGBM) predicts a bond's spread over a trailing 3-day market average. It uses the bond's coupon, maturity, rating, size, call and insurance features, the size and side of the trade, the bond's own last trade, and recent trades in similar bonds. Bond math turns the result into a price. Two more models predict the 10th and 90th percentiles to form the range, which we calibrate on held-out trades.

We compared against two non-AI approaches and kept both as baselines. The first is what a holder does today: look up the last trade. The second is a linear regression. The model's error is about half of either. For the explanation, we considered letting an LLM write it freely and decided against it. An LLM can produce a plausible reason that the model never used, and in a pricing tool that is worse than no reason. Instead, each sentence is built from the model's own feature contributions. Claude can optionally reword it, and the reworded version is thrown out if any number changes.

AI fits less well in two places. Bonds with no trade history in our data still miss by 2.76 points, so for them the tool mostly signals uncertainty. And our accuracy will trail SOLVE's, because their model uses dealer quotes we cannot get. Our main tradeoffs were building on simulated data first, so the pipeline existed before WRDS access came through, and publishing a no-install web demo next to the full app, at the cost of manual entry on the demo.

3. What we learned about AI through the work

The most useful lesson came from failing the same way twice. Tree models cannot predict values outside the range they were trained on. In week 4 we removed a calendar feature for that reason. We did not notice that our target had the same problem. The model predicted absolute yield, and as rates moved during the test window its errors drifted, from -0.05 to -0.12 yield points over eight weeks. Changing the target to spread over a market average let the average carry the rate level. Price error on the same test data fell from 0.85 to 0.64 points.

Two more lessons were about data and checks rather than the model. First, a calibration step can hide a broken model. Our v1 range covered 77% of trades, but only because the calibration aimed at 85%. Aimed honestly at 80%, it covered 71%. Second, data you generate yourself confirms whatever you built into it. On our first simulator, removing a bond's own trade history made the model slightly better. That is the opposite of the real market and of our premise, and it happened because the simulator let the model identify each bond from its rating and size. We rebuilt the simulator with more bond-specific variation and fewer trades for most bonds.

Working with Claude taught us to separate how confident an output sounds from whether it is right. Our v1 app, built with Claude, labeled buy and sell backwards relative to MSRB's convention, and that code read as cleanly as everything else. A test that a buyer should always pay more than a seller caught it. Context also changed results. Asking Claude to rebuild from our handoff doc would have produced new code we could not compare to v1, so we gave it the actual v1 code and every change stayed measurable. Its multiple-choice scoping questions forced us to make explicit decisions, and we overruled its recommendation of a local-only app because a link reaches more people.

4. Checking the work

We test against the success definition in section 1, on a period the model never saw. Against the last-trade lookup, the model roughly halves the error at every staleness level. Across four separate 30-day test windows, error is 0.71 ± 0.02 points and the 80% range covers 79.4% ± 0.4% of trades, so one lucky window does not explain the result. Automated tests check that the model behaves like the market and never uses a future trade. We also deployed the app publicly and used it. That is how we found its first load took over five minutes, which we fixed by shipping the trained model with the code.

Time since the bond last traded	Last-trade lookup	MuniPricer
Under 1 week	1.36	0.63
1 to 4 weeks	1.43	0.78
1 to 3 months	1.53	0.87
3 to 12 months	1.81	1.02

Average price error in points per 100 of face value, on simulated test trades.

The problems are real. Every number above comes from simulated trades, so it shows the method works, not that the tool is accurate in the market. No one who prices munis for a living has looked at an output yet. Part of the model's edge over the last-trade lookup is that it knows which side of the trade you are on, which a careful holder could partly adjust for by hand.

5. Work completed and plan for weeks 7 to 10

Done so far: a live app, a public repo with tests on every push, an evaluation report with a rolling backtest and ablations, a no-install web demo, and scripts that pull and clean MSRB trades from WRDS. We got WRDS access this week. The process packet shows how we got here, dead ends included.

Week 7: pull 12 months of MSRB trades, clean them, retrain, and record real numbers. Before the app serves real data, we restrict it to our team, since WRDS terms do not allow redistribution. Week 8: rerun the evaluation on real trades against the last-trade lookup, and show outputs to at least one muni practitioner through Booth alumni or faculty. Week 9: fix what the real errors and that feedback point to, and pick demo bonds that last traded months ago. Week 10: final presentation and write-up. We are splitting the remaining work into three roles: data, model, and write-up and demo.

Risks. If real data performs much worse, we report it and show where the model still beats the last-trade lookup. If WRDS lacks ratings or issue sizes, our loader already parses state and sector from bond descriptions, and we will look for a ratings file in WRDS. If no practitioner responds in time, we start outreach in week 7 and ask several people at once.

Source: MSRB, “Trading Patterns in the Municipal Securities Market,” March 2024. msrb.org